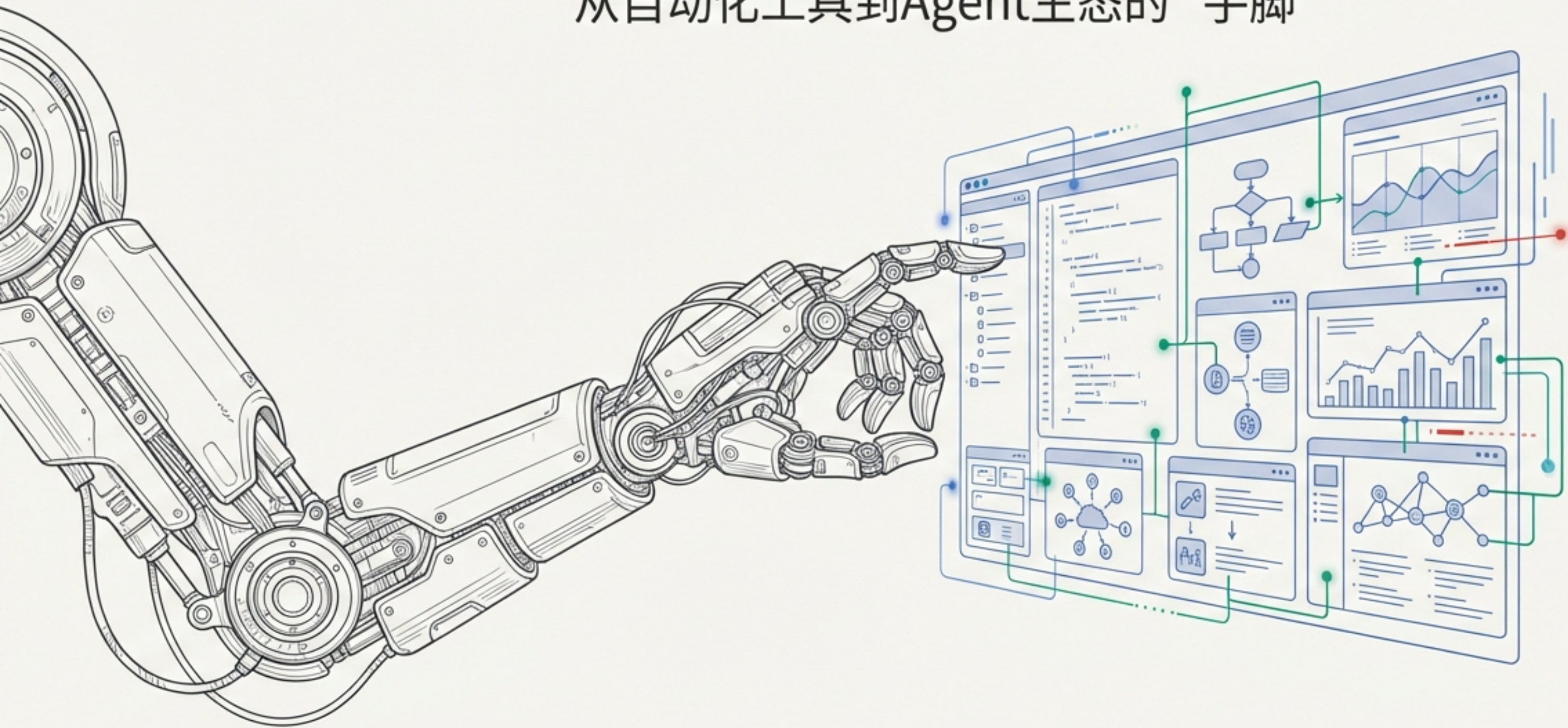


研发场景下的RPA应用实践与思考

从自动化工具到Agent生态的“手脚”



一把双刃剑：RPA的承诺与挑战



RPA的强大承诺

- **万物可达**：理论上可操作任何图形界面，无需侵入或改造原系统。
- **快速上手**：学习曲线平缓，能够快速构建自动化流程。



RPA的严峻现实

- **万物不稳**：对界面变化极其敏感，UI识别存在局限，维护成本高。
- **意外“救火”**：流程中断后，往往需要人工干预，反而增加负担。

我们面临的核心问题：如何驾驭RPA的“可达性”，同时规避其“不稳定性”，从而真正实现降本增效？

技术定位：RPA是“最后选项”，而非“优先选项”



RPA的角色

通过解析对机器最不友好的GUI，强行打通系统的“可达性”。

战略定位

当API等更稳定的接口不可用时，RPA是最后的备选方案（Last Resort），因为它本质上依赖最脆弱的交互层面。

我们的探索：三个典型研发场景的RPA实践

Excel数据处理



自动处理表格数据并根据规则上色。

组件成熟、逻辑精确、价值明确。



报告模板换色



在固定模板中进行确定性的点击和颜色替换。

逻辑简单、环境封闭、运行稳定。



图像平台采样



在一个无API的工程软件中，自动采样和处理图像。

无接口、识别困难、逻辑复杂。



初尝胜果：当RPA在正确的场景中发挥作用

项目一：Excel数据处理与上色

实现方式：利用成熟的Office标准接口组件构建，开发过程顺畅。

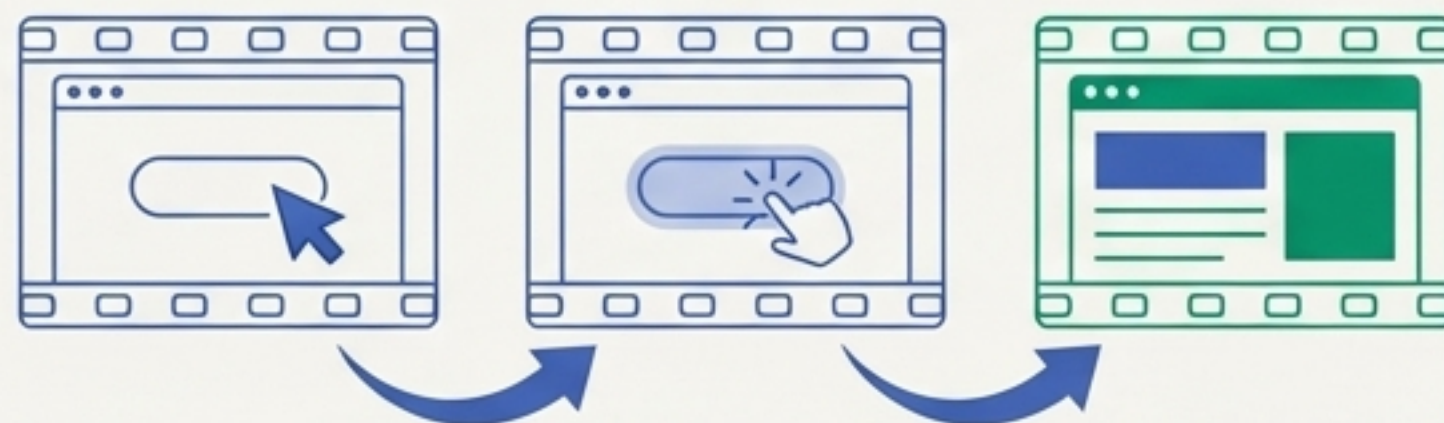
成果：实现100%自动化，流程精确可靠，极大释放了重复性工作的人力。



项目二：报告模板换色

实现方式：基于固定的点击坐标和顺序，流程呈线性，无复杂分支。

成果：即使涉及界面操作，但因环境封闭可控（无弹窗、无界面变化），运行极为稳定。



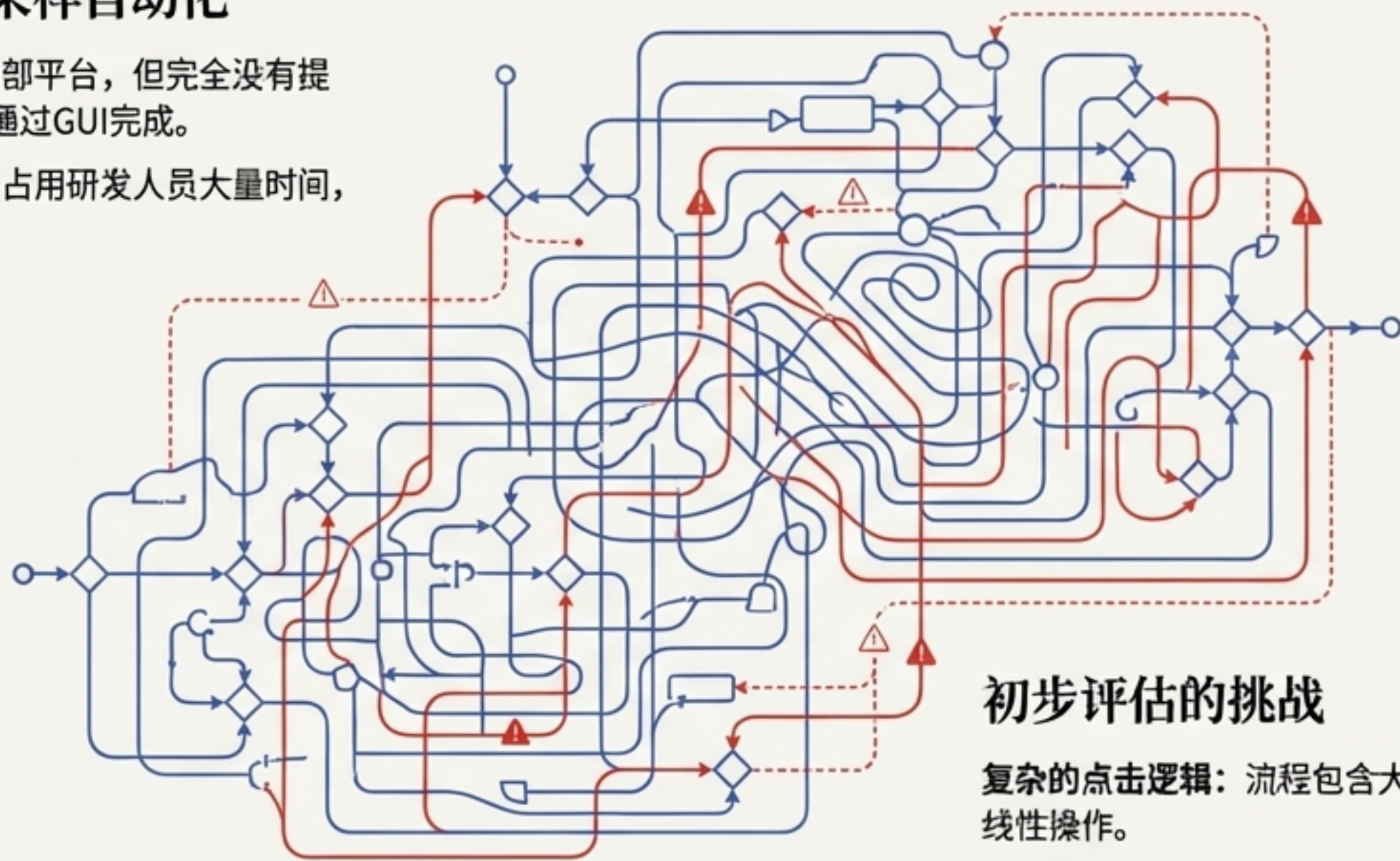
小结：在“组件可识别”和“逻辑确定”的场景下，RPA是无可争议的效率利器。

转折点：当“最后手段”遭遇“硬骨头”

项目背景：图像平台采样自动化

核心痛点：目标是一个关键的内部平台，但完全没有提供任何API接口，所有操作必须通过GUI完成。

业务需求：操作流程繁琐重复，占用研发人员大量时间，自动化需求迫切。



初步评估的挑战

复杂的点击逻辑：流程包含大量基于条件的判断与分支，远非线性操作。

非标的UI元素：平台为专用工程软件，其图标和控件的识别难度远高于标准办公软件（OCR/CV识别率低）。

高风险：流程一旦出错，可能导致采样数据错误，影响研发结论。

深入困境：我们在工程实践中遇到的具体技术瓶颈

1. 厂商能力瓶颈 (Vendor Capability Bottleneck)

细节：我们使用的Encoo RPA（云扩）主要面向办公软件场景，对研发类工程软件的底层组件支持严重不足。

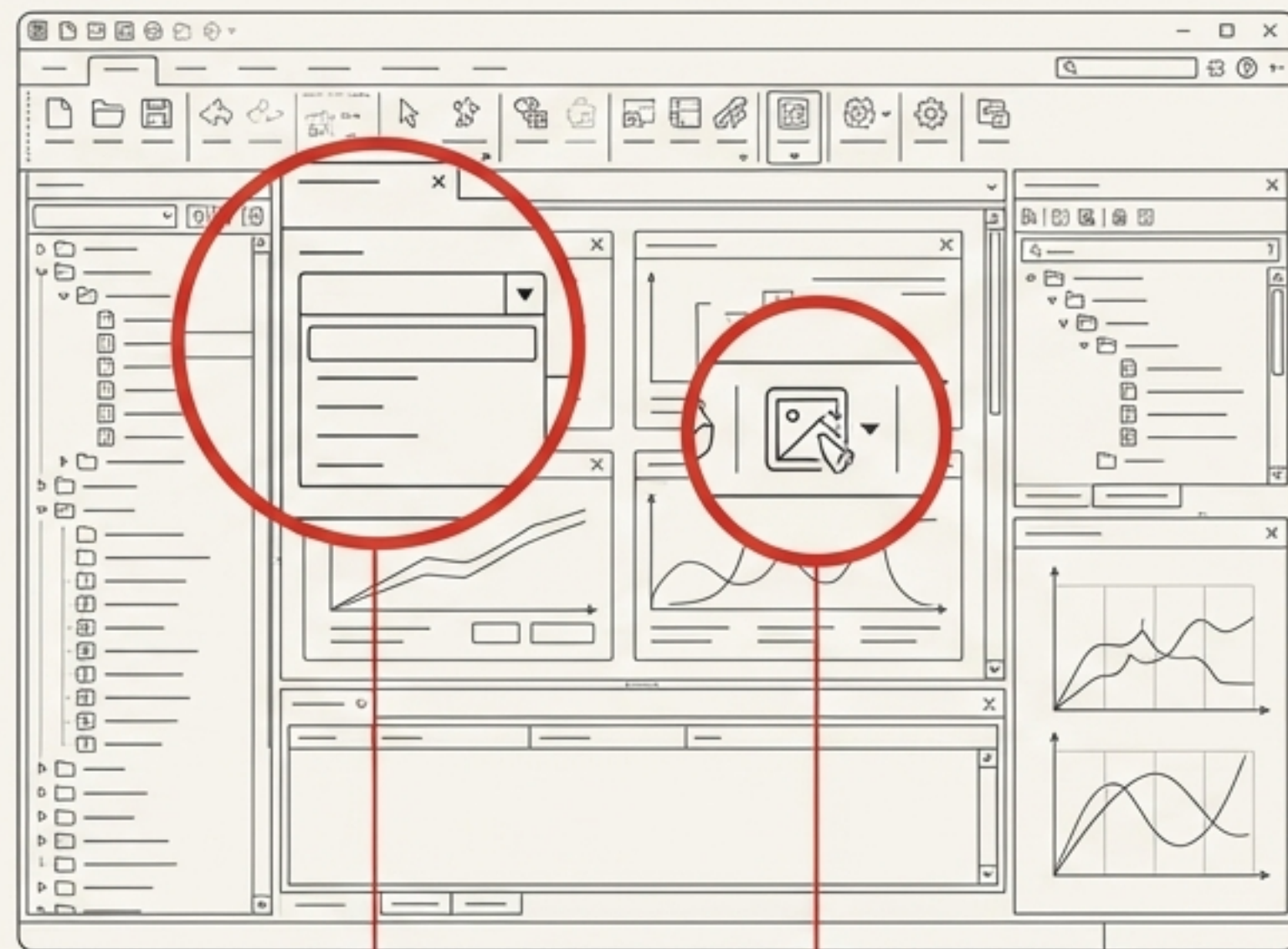
具体案例：“下拉框选择”组件在目标软件中完全无法识别，被迫降降级为模拟鼠标坐标点击的“笨办法”，极其脆弱。

2. UI识别的不稳定性 (Instability of UI Recognition)

细节：即使是看似简单的图标，也因渲染方式或状态变化而无法被稳定抓取，导致流程频繁中断。

3. 权限与环境干扰 (Permissions & Environmental Interference)

细节：RPA运行时需要独占鼠标和键盘，会中断用户正常工作。
解决方案：必须配置独立的窗口权限和控制台权限，甚至独立的虚拟机，这极大地增加了IT部署和维护的复杂度。



✗ 组件无法识别

✗ 状态变化导致抓取失败

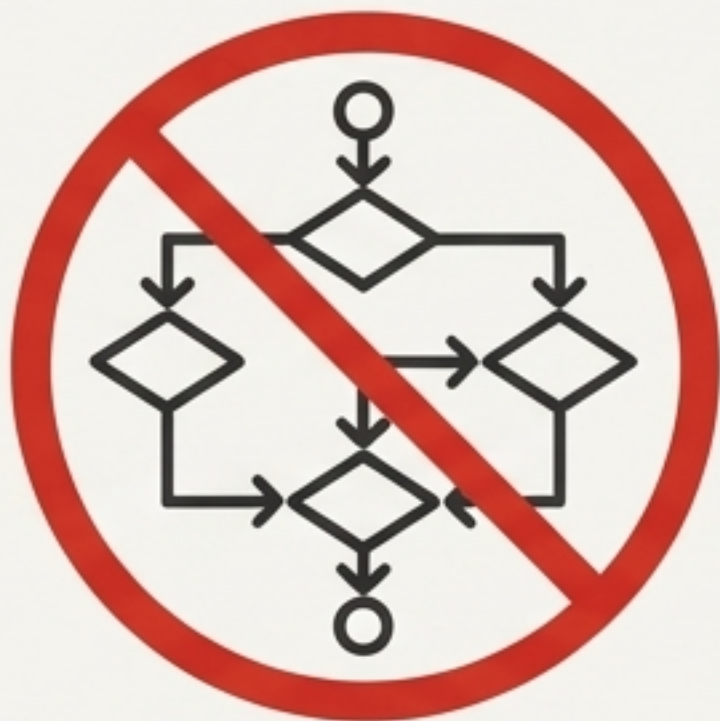
我们的突破：一个用于RPA决策的实践判断矩阵

将RPA重新定位为“简单、易上手的自动化工具”，而非一个全能的开发平台。

评估维度	 推荐RPA介入（Go）	 不推荐/高风险（No-Go）
1. UI交互复杂度	流程线性，点击位置和顺序固定，分支判断少。	逻辑分支多，依赖复杂的人工判断，流程非结构化。
2. 核心组件可识别性	使用标准控件（Office，Web表单，标准桌面应用）。	使用特殊工程软件、非标UI，或依赖OCR/CV识别。
3. 接口可达性	确定无任何API，且短期内不会开发。	系统已有API（应优先使用），或界面变动频繁。
4. 业务频率与负荷	高频、重复、规则确定的任务，人力负担重。	低频、偶发性需求，人工处理成本低于RPA开发维护成本。

结论：只有当一个需求在四个维度上都处于“ 推荐”区时，RPA才能成为真正降本增效的利器。

核心经验：我们总结的RPA项目“避坑”原则



原则一：避开复杂逻辑

如果业务流程包含大量条件跳转和分支判断，这已经超出了RPA的“舒适区”。

——不建议做。

原因：开发和维护成本会指数级增长，稳定性无法保证。



原则二：避开无法识别的UI

如果在关键步骤中，核心UI组件无法被RPA软件稳定识别（特别是工程软件），项目将寸步难行。

——不建议做。

原因：依赖坐标点击等脆弱方法，任何分辨率或窗口变化都可能导致失败。



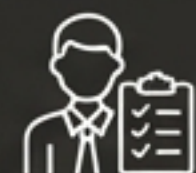
原则三：避开低频任务

如果任务使用频率低，人力负担不重，直接由人工完成更具成本效益。

——建议直接人工完成。

原因：RPA的开发、部署和长期维护成本，可能远高于节省的人力成本。

落地保障：建立清晰的角色分工与责任边界



需求方
(业务人员)

职责：梳理并提供标准作业程序（SOP）；明确业务价值与执行频率。

产出：标准化的业务流程文档。



RPA构建者
(流程工程师)

职责：依据“判断矩阵”进行技术可行性评估；负责脚本开发与异常处理（如弹窗）。

产出：可靠的自动化脚本；Go/No-Go评估结论。



RPA厂商
(Encoo)

职责：提供对特定软件（尤其是工程软件）的底层组件支持；解决核心识别技术问题。

产出：更强大的组件库和技术支持。



IT支持
(权限管理)

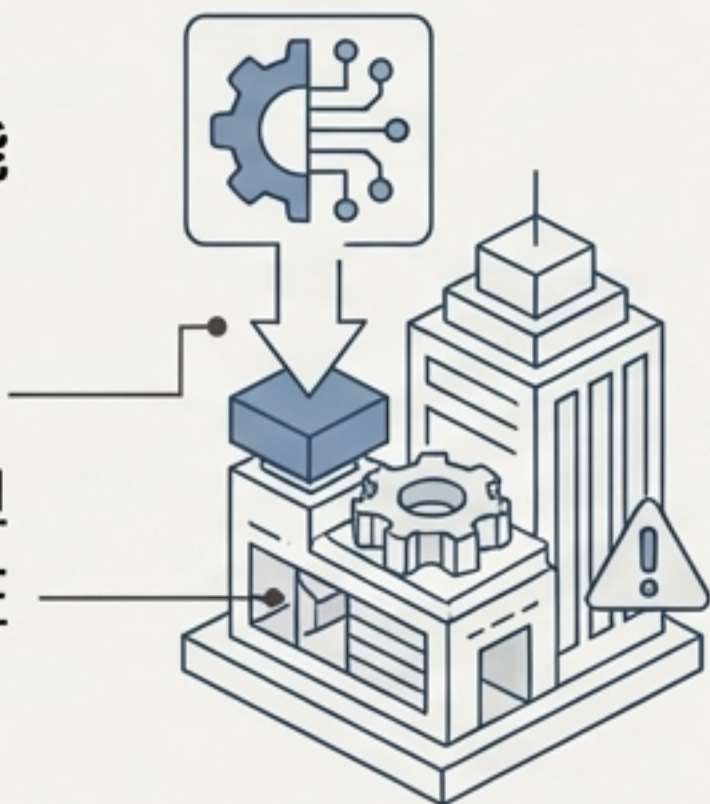
职责：配置独立的运行环境（控制台/虚拟机）；管理RPA运行所需的软件和数据权限。

产出：稳定、无干扰的RPA运行环境。

总结与展望：将RPA用在刀刃上

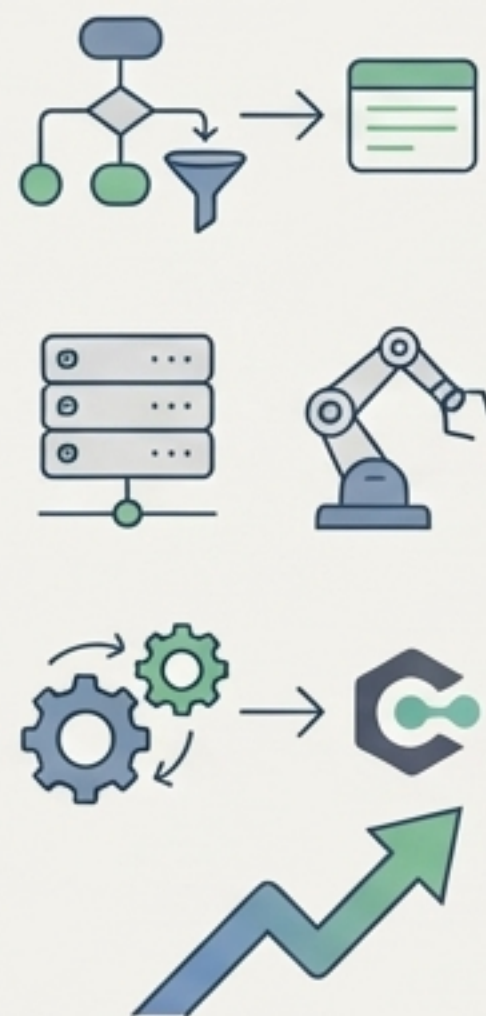
对现状的总结

- RPA是研发场景中一个强有力的**补充工具**，但绝非**万能钥匙**。
- 我们在办公类等“**确定性**”场景中已取得显著成效，但在**复杂工程软件**场景中验证了其**局限性**。



对未来的规划

- **流程上**：严格遵循“适用性判断矩阵”进行需求筛选，从源头避免陷入高成本的维护泥潭。
- **技术上**：推动IT部门建立独立的、标准化的RPA运行环境，确保自动化流程的稳定执行。
- **生态上**：持续向厂商（Encoo）反馈我们对工程软件的适配需求，共同推动工具能力的提升。



通过战略性的思考和筛选，我们可以让RPA回归其“自动化助手”的本质，创造真正的价值。